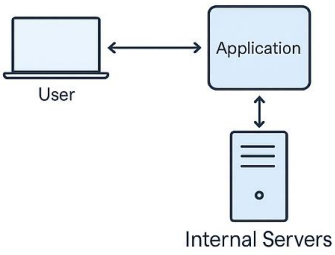
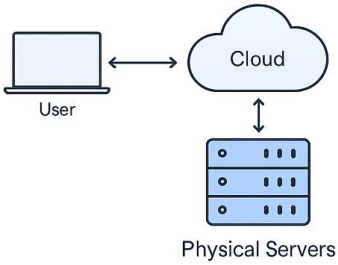
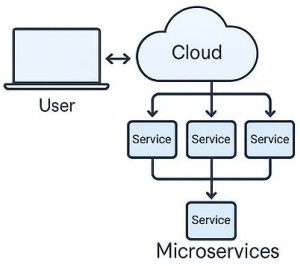
On-Prem	Cloud (Monolithic)	Cloud (Microservices)
On-Prem–Entire application hosted on company's internal servers 	Application in hosted in physical serves in placl servers 	Application in hosted in Cloud containers with microservices 
Performance and Reliability Tradeoff • High control over performance arreliability • Limited scalability and elasticity	**Performance and Reliability Tradeoff** • Moderate scalability with single-tier design • Potential performance bottlenecks	**Performance and Reliability Tradeoff** • High scalability and fault isolation • Increased complexity of management

On-Prem (All components in the company's data center)

When to consider

- Strict data-residency, compliance, or air-gapped environments.
- Existing investments in DC hardware and ops team.

Reference Architecture (logical)

- **Presentation**: Web (Agent & Policy Holder portal), Intranet (Company staff).
- **App tier**: Monolith (e.g., .NET or Java) running on VMs or bare-metal behind an on-prem load balancer.
- **Data tier**: Relational DB cluster (active/passive), file store/NAS for document uploads (medical reports, bills), enterprise queue (e.g., MSMQ/RabbitMQ) for async tasks.
- **Security**: Corporate IdP/AD for staff SSO; local auth for external users; WAF/Reverse proxy at perimeter.
- **Ops**: On-prem monitoring (e.g., Zabbix/SCOM), backups to secondary site; DR via warm standby.

Illustrative flow

- Agent submits Policy/Claim → App writes to DB + stores documents on NAS → Internal users review/approve via Intranet app → Notifications via SMTP/SMS gateway.

---

2) Cloud (Monolithic deployment on IaaS/PaaS; no microservices)

When to consider

- Fast migration from on-prem with minimal refactor.
- Predictable traffic with periodic spikes (e.g., renewal season).

Reference Architecture (logical)

- **Presentation**: Cloud WAF + CDN → App Service/VM Scale Set (single deployable).
- **Data tier**: Managed SQL (read replica) + Blob/Object Storage for uploads.
- **Integration**: Managed message queue for async (e.g., Azure Storage Queue/Service Bus, SQS).
- **Security**: Cloud IdP (B2C for external users, Entra ID/IdP for staff); Key Vault/Secrets Manager; Private endpoints.
- **Ops**: Auto-scale on CPU/queue length; multi-AZ HA; daily snapshots; backup vault.

Illustrative flow

- Agents/Policy Holders use internet portal → CDN/WAF → Monolith (App Service) → Managed SQL/Blob → Company staff Intranet app exposed via private access (VPN/Private Link).

---

3) Cloud (Microservices on containers)

When to consider

- Need for **independent scaling** of policy, claims, document, and notification domains.
- High change velocity; team autonomy; long-term cost/perf optimization.

Reference Architecture (logical)

- **API Gateway** in front of services.
- **Core services**: Policy, Customer, Claims, Document, Payments, Notification as separate services (containers on AKS/EKS/GKE).
- **Data**: Per-service databases (polyglot persistence); Object Storage for documents; search service for claims/policy lookup.
- **Async**: Event bus/stream (Service Bus/Kafka) for approvals, status changes; background workers.
- **Security**: Token-based auth (OIDC/JWT), mTLS/Network policies; secrets in Key Vault/Secret Manager.
- **Ops**: HPA for pods, circuit-breakers/retries, distributed tracing, blue/green or canary deployments, multi-region DR.

Illustrative flow

- User → WAF/CDN → API Gateway → respective service(s) → events on bus for downstream updates (e.g., “ClaimSubmitted” → “SentForApproval” → “Accepted/Rejected”).